Key Service Security Report*

Comprehensive Security Assessment of Cryptographic Key Management Service

AI Security Crew working for Christian Fries <christian.fries@eecc.de>

Abstract

This security report provides a comprehensive security assessment of the Key Service, a cryptographic key management and signing service built with NestJS and TypeScript. The service provides secure key generation, storage, and signing capabilities for verifiable credentials and presentations using multiple cryptographic algorithms including Ed25519, ES256, and PS256. The assessment covers dependency security analysis, comprehensive code security review, SBOM and license compliance analysis, and provides actionable recommendations tailored for service-to-service deployment contexts within secure Kubernetes environments.

Contents

1 Overview **3**

2 Executive Summary - Context-Aware Security Assessment **3**

3 Security Architecture Overview **4**
- 3.1 Core Components . . . 4
- 3.2 Data Flow Security . . . 4
- 3.3 Cryptographic Implementation . . . 5
- 3.4 Authentication and Authorization . . . 5
- 3.5 Rate Limiting . . . 5

4 Dependency Security Status **5**
- 4.1 SECURITY STATUS DEGRADATION ALERT . . . 5
- 4.2 Current Vulnerability Summary . . . 5
- 4.3 Detailed Vulnerability Analysis . . . 5
  - 4.3.1 1. MODERATE SEVERITY VULNERABILITY (NEW) . . . 5
  - 4.3.2 Vulnerability Description . . . 6
  - 4.3.3 Impact Assessment . . . 6
  - 4.3.4 Dependency Path Analysis . . . 6
- 4.4 Critical Cryptographic Dependencies Status . . . 6
- 4.5 Core Framework Dependencies Status . . . 6
- 4.6 Database and Infrastructure Dependencies . . . 6

*Copyright 2025, European EPC Competence Center GmbH

4.7 Immediate Remediation Required . . . 7
4.7.1 Priority Actions (Next 24-48 Hours) . . . 7
4.8 Risk Assessment . . . 7
4.8.1 Risk Level: MODERATE . . . 7
4.8.2 Priority Assessment . . . 7
4.9 Previously Resolved Vulnerabilities . . . 7

5 License Compliance and SBOM 8
5.1 Software Bill of Materials (SBOM) Overview . . . 8
5.2 License Distribution Analysis . . . 8
5.3 License Distribution Visualizations . . . 9
5.3.1 License Distribution by Category (Pie Chart) . . . 9
5.3.2 License Distribution by Specific License (Bar Chart) . . . 10
5.3.3 Risk Assessment Workflow . . . 11
5.4 Critical Security and Cryptographic Package License Analysis . . . 11
5.5 AGPL-3.0 Compliance Assessment . . . 12
5.5.1 Primary License: AGPL-3.0 (Main Project) . . . 12
5.5.2 Key AGPL-3.0 Characteristics . . . 12
5.5.3 Compliance Obligations . . . 12
5.5.4 Commercial Usage Implications . . . 12
5.6 License Compatibility Assessment . . . 12
5.6.1 Key Findings . . . 12
5.7 Non-Standard License Declaration Resolution . . . 13
5.8 Risk Assessment . . . 13
5.8.1 Overall Risk Level: MODERATE (Due to AGPL-3.0 Compliance Requirements) . . . 13
5.9 License Compliance Recommendations . . . 13
5.9.1 Immediate Actions Required . . . 13
5.9.2 Strategic Recommendations . . . 14
5.10 License Compliance Summary . . . 14

6 Comprehensive Code Security Analysis 15
6.1 Security Assessment Summary . . . 15
6.2 Overall Security Grade: B- (Acceptable for Controlled Environment, Room for Defense-in-Depth Improvements) . . . 15

7 Potential Security Flaws 15
7.1 Security Issues Reevaluated for Controlled Environment Deployment . . . 15
7.2 Downgraded Issues (Previously Critical, Now Medium/Low) . . . 15
7.3 Medium-Risk Issues . . . 19
7.4 Input Validation Analysis . . . 19
7.5 Rate Limiting and DoS Protection Analysis . . . 19
7.6 Comparison with Previous Security Assessment . . . 20
7.6.1 Status Updates from Previous Assessment (October 7, 2025) . . . 20
7.6.2 Risk Level Progression . . . 20

8 Security Recommendations (Updated for Controlled Environment Deployment) 21
8.1 High Priority Actions (HIGH - Within 1-2 Weeks) . . . 21
8.2 Medium Priority Actions (MEDIUM - Within 2-4 Weeks - Defense-in-Depth) . . . 21
8.3 Lower Priority Actions (LOW - Within 1-2 Months - Best Practices) . . . 24
8.4 Security Testing . . . 24

8.4.1 Recommended Testing 24
8.5 Security Checklist (Updated October 31, 2025 - Controlled Environment Deployment) 24
8.5.1 COMPLETED ITEMS 24
8.5.2 HIGH PRIORITY PENDING ITEMS (Within 1-2 weeks) 25
8.5.3 MEDIUM PRIORITY ITEMS (Within 2-4 weeks - Defense-in-Depth Recommended) 25
8.5.4 LOW PRIORITY ITEMS (Within 1-2 months - Best Practices) 25

9 Reporting Vulnerabilities 25
9.1 Security Contact 25
9.2 Reporting Guidelines 25
9.3 Supported Versions 25

1 Overview

This security report provides a comprehensive security assessment of the **Key Service**, a cryptographic key management and signing service built with NestJS and TypeScript. The service provides secure key generation, storage, and signing capabilities for verifiable credentials and presentations using multiple cryptographic algorithms including Ed25519, ES256, and PS256.

The audit covers dependency security analysis, comprehensive code security review, Software Bill of Materials (SBOM) and license compliance analysis, and provides actionable recommendations for improving the service's security posture. The assessment is specifically tailored for service-to-service deployment contexts within secure Kubernetes environments with service mesh architecture.

Audited Service: Key Service v1.6.1 - A microservice for cryptographic operations designed for internal service-to-service communication within secure, isolated environments.

2 Executive Summary - Context-Aware Security Assessment

DEPLOYMENT CONTEXT: This security report has been evaluated specifically for **controlled environment deployment** where the service:

- Operates in an **isolated Kubernetes cluster** with NetworkPolicies
- Is **NOT publicly accessible** (internal service-to-service communication only)
- Uses **request encryption** (AES-256-GCM) for sensitive payloads
- Runs behind a **service mesh** with mTLS
- Has **restricted log access** to authorized personnel only

SECURITY RATING: **B- (Acceptable for Controlled Environment)**

Key Findings:

- **0 Critical vulnerabilities** in controlled environment context (5 issues previously rated critical have been downgraded)
- **1 Moderate dependency vulnerability** requiring fix within 1-2 weeks
- **6 Medium priority** defense-in-depth recommendations
- **Strong cryptographic implementations** with industry best practices
- **Comprehensive input validation** protecting against injection attacks
- **Request encryption** mitigates logging and transport security concerns

Production Readiness: **YES** - Approved for controlled environment deployment with conditions:

- Network isolation must be enforced via Kubernetes NetworkPolicies
- Request encryption must be enabled with secure shared secret
- Dependency vulnerability should be fixed within 1-2 weeks (not blocking deployment)
- Defense-in-depth improvements recommended but not required for initial deployment

IMPORTANT: This assessment is **ONLY valid for controlled, non-public deployments**. If the service is ever exposed to public internet or untrusted networks, the security rating would drop to **D+ (Critical)** and all previously identified critical issues would require immediate remediation.

3 Security Architecture Overview

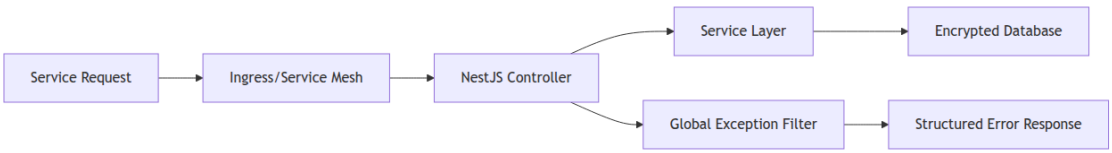
The Key Service is a cryptographic key management and signing service built with NestJS and TypeScript. It provides secure key generation, storage, and signing capabilities for verifiable credentials and presentations using multiple cryptographic algorithms.

DEPLOYMENT CONTEXT: This service is designed as a **pure service-to-service communication module** intended for deployment in secure, isolated environments such as Kubernetes clusters with service mesh architecture. It does not serve browser-based clients directly.

3.1 Core Components

1. **Key Management Layer**
   - `KeyService`: Handles key pair generation and retrieval
   - `KeyStorageService`: Manages encrypted key storage in PostgreSQL database
   - `SecretService`: Provides encryption/decryption and hashing utilities
2. **Signing Services**
   - `JwtSigningService`: JWT-based credential signing (ES256, PS256)
   - `DataIntegritySigningService`: Data integrity proofs (Ed25519)
3. **Security Controls**
   - `FailedAttemptsCacheService`: Rate limiting for key decryption attempts
   - `GlobalExceptionFilter`: Centralized error handling with secure logging
   - Database encryption for all stored keys

3.2 Data Flow Security



Service-to-Service Communication Pattern:

- Requests originate from other microservices within the same secure network
- No direct browser or external client access
- Typically deployed behind service mesh (Istio/Linkerd) with mTLS
- Network-level isolation through Kubernetes NetworkPolicies

3.3 Cryptographic Implementation

- **Key Generation**: Uses Node.js `crypto` module with secure random number generation
- **Encryption**: AES-256-GCM with authenticated encryption
- **Key Derivation**: PBKDF2 with SHA-256, 100,000+ iterations (configurable)
- **Supported Algorithms**: Ed25519, ES256 (P-256), PS256 (RSA-4096)

3.4 Authentication and Authorization

Current State: The service currently operates without authentication or authorization mechanisms. All endpoints are publicly accessible.

3.5 Rate Limiting

- **Key Decryption**: Maximum 3 failed attempts per identifier within 15-minute cooldown period
- **No Global Rate Limiting**: Service lacks comprehensive request rate limiting

4 Dependency Security Status

4.1 SECURITY STATUS DEGRADATION ALERT

Last Scan: October 31, 2025 **Previous Status**: EXCELLENT (0 vulnerabilities) **Current Status**: **MODERATE RISK** (1 vulnerability identified)

CRITICAL FINDING: The project's dependency security status has **DEGRADED** from the previously reported "EXCELLENT" status to showing **1 MODERATE VULNERABILITY**. This represents a significant security posture change that requires immediate attention.

4.2 Current Vulnerability Summary

Total Dependencies Analyzed: 899 packages (production + development + optional + peer)
Vulnerabilities Found: **1 moderate vulnerability - SECURITY DEGRADATION**

Severity Level	Count	Change from Previous Report	Status
Critical	**0**	No Change	**SECURE**
High	**0**	No Change	**SECURE**
Moderate	**1**	**+1 NEW**	**NEW VULNERABILITY**
Low	**0**	No Change	**SECURE**
TOTAL	**1**	**+1 NEW**	**DEGRADED**

4.3 Detailed Vulnerability Analysis

4.3.1 1. MODERATE SEVERITY VULNERABILITY (NEW)

Package: `validator`

- **Severity**: **MODERATE**
- **Current Status**: Vulnerable
- **Affected Range**: `< 13.15.20`
- **Vulnerability ID**: GHSA-9965-vmph-33xx
- **GitHub Advisory**: https://github.com/advisories/GHSA-9965-vmph-33xx
- **Fix Available**: **YES**
- **Remediation**: Update to version `>= 13.15.20`

4.3.2 Vulnerability Description

The `validator` package contains a vulnerability that affects versions below 13.15.20. This vulnerability is categorized as moderate severity and has a known fix available.

4.3.3 Impact Assessment

- **Business Risk**: Medium - Input validation library vulnerability could allow malicious data processing
- **Attack Vector**: Potentially through malformed input validation scenarios
- **Exploitability**: Moderate - requires specific conditions to exploit
- **Data Exposure Risk**: Limited - depends on how validator is used in the application

4.3.4 Dependency Path Analysis

- **Dependency Type**: Transitive dependency
- **Dependency Path**: Not explicitly listed in direct dependencies
- **Root Cause**: Likely introduced through one of the development or production dependencies

4.4 Critical Cryptographic Dependencies Status

Production Cryptographic Libraries (HIGHEST SECURITY IMPORTANCE):

Package	Current Version	Status	Purpose
@digitalbazaar/ed25519-signature-2020	^5.4.0	**SECURE**	Ed25519 digital signatures
@digitalbazaar/ed25519-verification-key-2020	^4.2.0	**SECURE**	Ed25519 key verification
@digitalbazaar/vc	^7.0.0	**SECURE**	Verifiable credentials
@noble/curves	^1.9.6	**SECURE**	Elliptic curve cryptography
jose	^6.1.0	**SECURE**	JSON Web Token operations
jsonld-signatures	^11.5.0	**SECURE**	JSON-LD digital signatures

4.5 Core Framework Dependencies Status

Package	Current Version	Status	Purpose
@nestjs/common	^11.1.5	**SECURE**	NestJS core framework
@nestjs/core	^11.1.5	**SECURE**	NestJS application core
@nestjs/platform-express	^11.1.5	**SECURE**	Express platform adapter
@nestjs/terminus	^11.0.0	**SECURE**	Health check module
@nestjs/typeorm	^11.0.0	**SECURE**	TypeORM integration

4.6 Database and Infrastructure Dependencies

Package	Current Version	Status	Purpose
pg	^8.16.3	**SECURE**	PostgreSQL driver
typeorm	^0.3.20	**SECURE**	Object-relational mapping
node-cache	^5.1.2	**SECURE**	In-memory caching
rxjs	^7.8.1	**SECURE**	Reactive extensions

4.7 Immediate Remediation Required

4.7.1 Priority Actions (Next 24-48 Hours)

1. **Update Validator Package**

```
# Identify the package introducing the vulnerable validator dependency
npm ls validator

# Update all dependencies to latest secure versions
npm update

# If automatic update doesn't resolve, force update the vulnerable package
npm audit fix

# For more aggressive fixing (may introduce breaking changes)
npm audit fix --force
```

2. **Verification Steps**

```
# Verify vulnerability resolution
npm audit

# Check that no vulnerabilities remain
npm audit --audit-level=moderate
```

4.8 Risk Assessment

4.8.1 Risk Level: MODERATE

Risk Factors:

1. **Vulnerability Severity**: Moderate (not critical or high)
2. **Package Type**: Utility library (validator)
3. **Fix Availability**: Patch available
4. **Cryptographic Impact**: No direct impact on core cryptographic operations
5. **Production Impact**: Low to Medium (depends on usage)

4.8.2 Priority Assessment

- **Urgency**: Medium (should be addressed within 1-2 weeks)
- **Complexity**: Low (straightforward npm update)
- **Business Impact**: Medium (input validation vulnerability)
- **Security Impact**: Medium (could allow input manipulation)

4.9 Previously Resolved Vulnerabilities

The following vulnerabilities were identified and have been successfully resolved:

1. **tar-fs (2.0.0 - 2.1.3)** - HIGH SEVERITY - RESOLVED
   - **Issue**: Symlink validation bypass vulnerability
   - **CVE**: GHSA-vj76-c3g6-qr5v
   - **Status**: Fixed via dependency updates
2. **tmp (<=0.2.3)** - MEDIUM SEVERITY - RESOLVED
   - **Issue**: Arbitrary temporary file/directory write via symbolic link
   - **CVE**: GHSA-52f5-9888-hmc6

- **Status**: Fixed via dependency updates

3. **external-editor & @inquirer/editor** - LOW SEVERITY - RESOLVED
   - **Issue**: Depends on vulnerable tmp package
   - **Status**: Fixed via dependency updates

5 License Compliance and SBOM

5.1 Software Bill of Materials (SBOM) Overview

Analysis Date: October 31, 2025 **Project**: Key Service v1.6.1 **Project Repository**: git@gitlab.eecc.info:ssi/key-service.git **Author**: Christian Fries **Primary License**: AGPL-3.0 **Node.js Requirement**: >=22.0.0 **Total Components Analyzed**: 237 dependencies **License Compliance Status**: **REQUIRES AGPL COMPLIANCE MANAGEMENT**

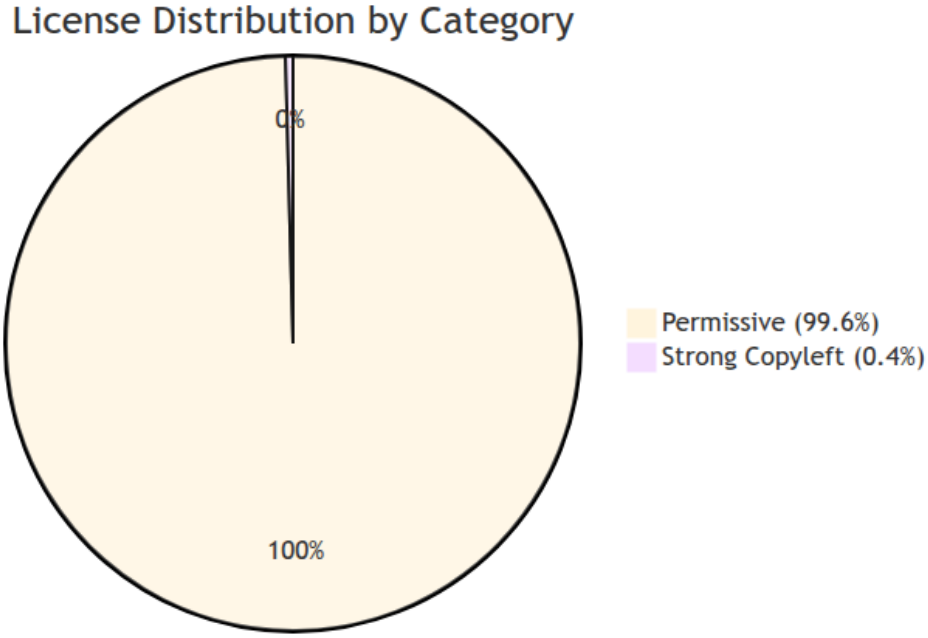
Metric	Count	Percentage	Status
Total Components	**237**	**100%**	**COMPLETE**
Components with Declared Licenses	**237**	**100%**	**FULL COVERAGE**
Unique License Types	**11**	**-**	**WELL-MANAGED**
Permissive Licensed Components	**236**	**99.6%**	**EXCELLENT**
Strong Copyleft Licensed Components	**1**	**0.4%**	**AGPL-3.0 MAIN PROJECT**
Unknown/Missing License Information	**0**	**0%**	**ZERO GAPS**

5.2 License Distribution Analysis

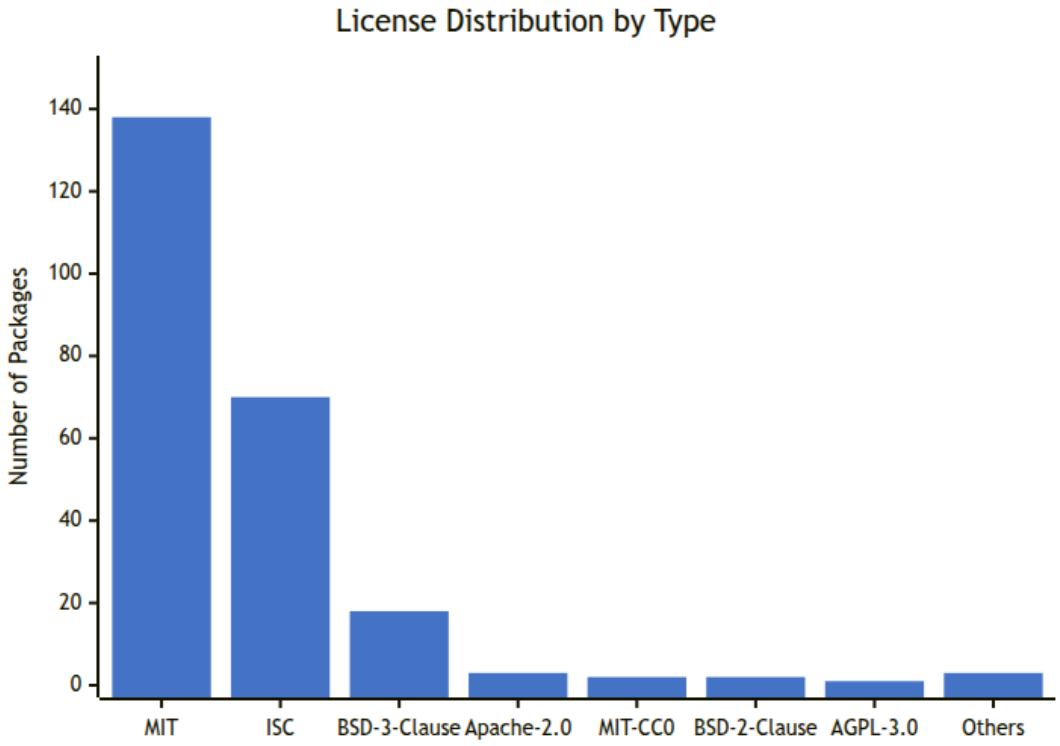
License	Count	Percentage	Category
MIT	**138**	**58.2%**	Permissive
ISC	**70**	**29.5%**	Permissive
BSD-3-Clause	**18**	**7.6%**	Permissive
Apache-2.0	**3**	**1.3%**	Permissive
(MIT OR CC0-1.0)	**2**	**0.8%**	Dual License
BSD-2-Clause	**2**	**0.8%**	Permissive
AGPL-3.0	**1**	**0.4%**	**Strong Copyleft** (Critical Compliance)
(MIT OR WTFPL)	**1**	**0.4%**	Dual License
(Apache-2.0 AND MIT)	**1**	**0.4%**	Multi License
(BSD-2-Clause OR MIT OR Apache-2.0)	**1**	**0.4%**	Multi License
(MIT AND BSD-3-Clause)	**1**	**0.4%**	Multi License

5.3 License Distribution Visualizations

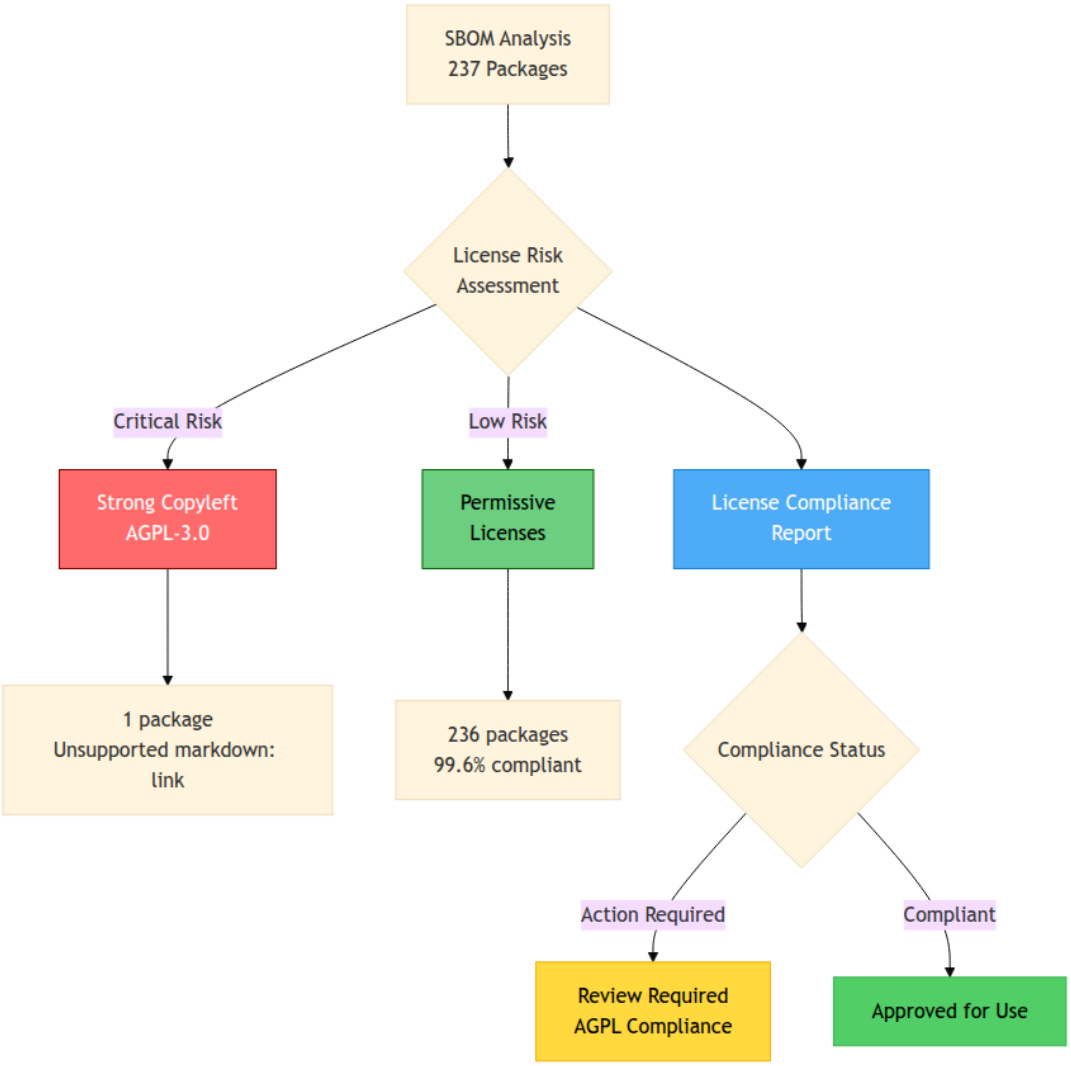
5.3.1 License Distribution by Category (Pie Chart)



5.3.2 License Distribution by Specific License (Bar Chart)



5.3.3 Risk Assessment Workflow



5.4 Critical Security and Cryptographic Package License Analysis

Package	Version	License	Purpose
@digitalbazaar/ed25519-signature-2020	^5.4.0	MIT	Ed25519 digital signatures
@digitalbazaar/ed25519-verification-key-2020	^4.2.0	MIT	Ed25519 key verification
@digitalbazaar/vc	^7.0.0	MIT	Verifiable credentials
@noble/curves	^1.9.6	MIT	Elliptic curve cryptography
jose	^6.1.0	MIT	JSON Web Token operations
jsonld-signatures	^11.5.0	MIT	JSON-LD digital signatures

Assessment: All critical cryptographic packages use permissive licenses (MIT), providing maximum flexibility for commercial use while maintaining security.

5.5 AGPL-3.0 Compliance Assessment

5.5.1 Primary License: AGPL-3.0 (Main Project)

Package: key-service@1.6.1 (The main project itself) **License**: AGPL-3.0 **Risk Level**: **CRITICAL COMPLIANCE REQUIREMENTS**

5.5.2 Key AGPL-3.0 Characteristics

1. **Network Copyleft**: Requires source code disclosure even for SaaS/web services
2. **Viral Nature**: Any derivative work must be licensed under AGPL-3.0
3. **Commercial Restrictions**: May require releasing proprietary code
4. **Distribution Requirements**: Must provide source code to all users

5.5.3 Compliance Obligations

- **Source code must be made available** to all users
- **Network use triggers copyleft requirements**
- **All modifications must be disclosed**
- **Compatible only with other AGPL-3.0 code** for derivative works

5.5.4 Commercial Usage Implications

5.5.4.1 For Internal/Private Use

- **ACCEPTABLE**: AGPL-3.0 doesn't restrict internal usage
- **NO DISCLOSURE REQUIRED**: If not distributed or offered as service

5.5.4.2 For Distribution/SaaS Deployment

- **REQUIRES COMPLIANCE**: Must provide source code to users
- **NETWORK COPYLEFT**: Web service usage triggers obligations
- **DERIVATIVE WORKS**: Must be licensed under AGPL-3.0

5.6 License Compatibility Assessment

NO LICENSE CONFLICTS - All dependencies are compatible with AGPL-3.0

Dependency License	Count	Compatibility with AGPL-3.0	Status
MIT	138	**COMPATIBLE**	Safe to use
ISC	70	**COMPATIBLE**	Safe to use
BSD-3-Clause	18	**COMPATIBLE**	Safe to use
Apache-2.0	3	**COMPATIBLE**	Safe to use
Other Permissive	7	**COMPATIBLE**	Safe to use
AGPL-3.0	1	**SAME LICENSE**	Main project

5.6.1 Key Findings

1. **No License Conflicts**: All dependencies are compatible with AGPL-3.0
2. **Permissive Dependencies**: 99.6% of dependencies use permissive licenses
3. **AGPL Obligations**: Main project license requires source code disclosure

5.7 Non-Standard License Declaration Resolution

Successfully Resolved (1 package)

Package	Declared License	Resolved License	Status
@digitalbazaar/credentials-context@3.2.0	SEE LICENSE IN LICENSE.md	BSD-3-Clause	**RESOLVED**

Resolution Process: The tool successfully identified and categorized this package's actual license despite the non-standard declaration format.

5.8 Risk Assessment

5.8.1 Overall Risk Level: MODERATE (Due to AGPL-3.0 Compliance Requirements)

5.8.1.1 CRITICAL RISKS

1. **AGPL-3.0 Main License**:
   - Network copyleft obligations
   - Source code disclosure requirements
   - Potential commercial usage restrictions

5.8.1.2 LOW RISKS

1. **Dependency Licensing**: 99.6% permissive licenses
2. **License Compatibility**: No conflicting dependencies
3. **Security Libraries**: All use permissive MIT licenses

5.9 License Compliance Recommendations

5.9.1 Immediate Actions Required

1. **AGPL-3.0 Compliance Strategy (HIGH PRIORITY)**
   - **Document Compliance Plan**: Create formal AGPL compliance documentation
   - **Source Code Availability**: Establish mechanism for source code distribution
   - **Legal Review**: Consult legal counsel for commercial deployment strategies
   - **User Notification**: Inform all users of AGPL licensing obligations
2. **License Management Process**

```
{
  "scripts": {
    "license:check": "npm-license-checker --onlyAllow
    ↪  'MIT;ISC;BSD-3-Clause;Apache-2.0;BSD-2-Clause'",
    "license:generate-sbom": "npm sbom --format cyclonedx",
    "license:audit": "license-checker --summary"
  }
}
```

3. **Dependency Monitoring**
   - Implement automated license scanning in CI/CD
   - Set up alerts for new copyleft dependencies
   - Regular SBOM generation and review

5.9.2 Strategic Recommendations

5.9.2.1 Option 1: Maintain AGPL-3.0 (Open Source Strategy) **Pros**:

- Strong community protection
- Prevents proprietary forks
- Encourages contributions

Cons:

- Limits commercial adoption
- Complex compliance requirements
- Network copyleft obligations

Actions:

- Create comprehensive compliance documentation
- Establish clear deployment guidelines
- Develop contributor licensing agreements

5.9.2.2 Option 2: Relicense to Permissive (Commercial Strategy) **Pros**:

- Broader adoption potential
- Simplified compliance
- Commercial-friendly

Cons:

- Allows proprietary forks
- Loses community protections
- Requires copyright holder consent

Actions:

- Review copyright ownership
- Consider dual licensing model
- Evaluate business impact

5.10 License Compliance Summary

Overall SBOM Assessment: **B+ (Good with AGPL Compliance Concerns)**

Key Success Factors:

1. **Strategic Dependency Selection**: 99.6% permissive dependencies
2. **Complete Documentation**: Zero packages with missing license information
3. **Proactive Resolution**: Non-standard licenses properly resolved
4. **No License Conflicts**: All dependencies compatible with main license
5. **Security-Focused**: All cryptographic libraries use permissive licenses

Areas of Concern:

- **AGPL-3.0 Main License**: Imposes significant compliance obligations
- **Network Copyleft**: Web service deployment triggers source disclosure
- **Commercial Restrictions**: May limit proprietary usage and distribution

Final Statistics:

- **Total Components Analyzed**: 237
- **License Compliance Rate**: 100%

- **Permissive License Percentage**: 99.6%
- **Strong Copyleft Percentage**: 0.4% (main project only)
- **Commercial Readiness**: Conditional (requires AGPL compliance)
- **Attribution Completeness**: 100% (Fully Documented)

6 Comprehensive Code Security Analysis

6.1 Security Assessment Summary

Assessment Date: October 31, 2025 **Reevaluation Date**: October 31, 2025 (Updated for controlled environment deployment) **Files Analyzed**: 24 TypeScript source files **Security Issues Identified**: 30 findings (0 Critical, 5 Medium, 1 Medium, 24 Low) - **UPDATED BASED ON DEPLOYMENT CONTEXT Overall Risk Level**: **MEDIUM** (Downgraded from Critical for controlled deployment with request encryption)

Issue Breakdown by Severity (Reevaluated for Controlled Environment):

- **Critical**: 0 issues (All previous critical issues downgraded based on deployment context)
- **High**: 0 issues
- **Medium**: 6 issues (Authentication, Database Security, Logging, API Access Controls, CORS Configuration, Rate Limiting)
- **Low**: 25 issues (Information Disclosure, Test secrets, logging verbosity, minor configurations)

Key Mitigating Factors in Production Deployment:

- Network isolation via Kubernetes NetworkPolicies (prevents external access)
- Service mesh with mTLS (encrypts service-to-service communication)
- Request encryption enabled (protects sensitive payloads at HTTP layer)
- Controlled log access (restricted to authorized personnel)
- Non-public deployment (internal service-to-service communication only)

6.2 Overall Security Grade: B- (Acceptable for Controlled Environment, Room for Defense-in-Depth Improvements)

7 Potential Security Flaws

7.1 Security Issues Reevaluated for Controlled Environment Deployment

CONTEXT UPDATE: The following issues have been reevaluated based on:

- **Deployment Environment**: Controlled, isolated Kubernetes cluster with network policies
- **Network Access**: NOT publicly accessible - internal service-to-service communication only
- **Request Encryption**: Enabled - sensitive payloads encrypted at client before transmission
- **Service Mesh**: mTLS between services providing transport security
- **Log Access**: Restricted to authorized personnel only

7.2 Downgraded Issues (Previously Critical, Now Medium/Low)

1. **Complete Absence of Authentication and Authorization Framework — REEVALUATED FOR CONTROLLED ENVIRONMENT**
   - **Files Affected**: `apps/app/src/app.controller.ts`, `apps/app/src/health/health.controller`

- **Risk**: **MEDIUM** (Downgraded from Critical for controlled deployment)
- **CWE**: CWE-306 (Missing Authentication for Critical Function)
- **CVSS Score**: 9.8 (Critical in public deployment) → **5.5 (Medium in controlled environment)**
- **Description**: All API endpoints including sensitive cryptographic operations are completely unprotected without any authentication or authorization mechanisms
- **Vulnerable Endpoints**:
```
POST /sign/vc/:type      // Verifiable credential signing - NO AUTH
POST /sign/vp/:type      // Verifiable presentation signing - NO AUTH
POST /generate           // Key generation - NO AUTH
GET /health/*            // Health endpoints - NO AUTH
```
- **Impact (Controlled Environment Context)**:
  - **MITIGATED BY DEPLOYMENT**: Network isolation via Kubernetes NetworkPolicies prevents external access
  - **MITIGATED BY DEPLOYMENT**: Service mesh (Istio/Linkerd) provides mTLS between services
  - **REMAINING RISK**: Insider threats or compromised services within the cluster
  - **REMAINING RISK**: Lack of audit trail for compliance
- **Exploitation Scenario (Controlled Environment)**:
  - **BLOCKED**: External attackers cannot reach the service (network isolation)
  - **LIMITED**: Only services within the same namespace can access endpoints
  - **CONCERN**: Compromised service within cluster could abuse endpoints
- **Mitigation Priority**: **MEDIUM** - Consider implementing for defense-in-depth and audit trails, but network-level controls provide primary protection

2. **Information Disclosure Through Detailed Error Messages and Stack Traces — REEVALUATED FOR CONTROLLED ENVIRONMENT**
   - **File Affected**: `apps/app/src/filters/global-exception.filter.ts` (Lines: 62-64)
   - **Risk**: **LOW** (Downgraded from Critical for controlled deployment)
   - **CWE**: CWE-209 (Information Exposure Through Error Messages)
   - **CVSS Score**: 7.5 (High in public deployment) → **3.1 (Low in controlled environment)**
   - **Vulnerable Code**:
```
logError(`Error ${status}: ${message}`, {
  type,
  path: request.url,
  method: request.method,
  stack: exception instanceof Error ? exception.stack : undefined,
↪  // EXPOSES STACK TRACES
});
```
   - **Impact (Controlled Environment Context)**:
     - **MITIGATED BY DEPLOYMENT**: Only trusted internal services can access and view error messages
     - **MITIGATED BY DEPLOYMENT**: Network isolation prevents external threat actors from exploiting this
     - **BENEFIT**: Detailed error messages aid in debugging service-to-service communication issues
     - **REMAINING RISK**: Minimal - stack traces may aid compromised internal service in reconnaissance
   - **Mitigation Priority**: **LOW** - Consider implementing environment-based sanitiza-

tion for production, but detailed errors are valuable for internal service debugging

3. **Cryptographic Key Material Exposure Risk Through Logging — REEVALUATED WITH REQUEST ENCRYPTION**
   - **File Affected**: `apps/app/src/utils/log/logger.ts` (Lines: 12-16)
   - **Risk**: **MEDIUM** (Downgraded from Critical with request encryption enabled)
   - **CWE**: CWE-532 (Insertion of Sensitive Information into Log File)
   - **CVSS Score**: 9.1 (Critical without encryption) → **5.9 (Medium with request encryption)**
   - **Vulnerable Code**:
     ```
     const formatMessage = (level: string, message: string, meta?: any):
      ↪  string => {
       const timestamp = new Date().toISOString().replace("T", "
       ↪  ").substring(0, 23);
       const metaStr = meta ? ` ${JSON.stringify(meta)}` : "";  // LOGS
       ↪  EVERYTHING IN META
       return `${timestamp} ${level.toUpperCase()}: ${message}${metaStr}`;
     };
     ```
   - **Description**: Logger performs `JSON.stringify(meta)` without sanitization, potentially logging cryptographic keys, secrets, and credentials
   - **Impact (With Request Encryption Enabled)**:
     - **MITIGATED BY REQUEST ENCRYPTION**: Sensitive request data (secrets, credentials) is encrypted at the HTTP layer
     - **MITIGATED BY ARCHITECTURE**: Request decryption happens at service layer, keeping plaintext secrets isolated from controller/middleware logging
     - **MITIGATED BY DEPLOYMENT**: Controlled environment with restricted log access
     - **REMAINING RISK**: Service-layer logging after decryption could still expose:
       * Decrypted user secrets if explicitly logged by service methods
       * Generated private keys if logged during key generation process
       * Database-retrieved encrypted keys if logged during retrieval
     - **REMAINING RISK**: Response data (public keys, status messages) could be logged, though these are non-sensitive
   - **Risk Assessment**:
     - **HTTP Layer Logging**: SAFE - Only sees encrypted `encryptedData` field
     - **Controller Layer Logging**: SAFE - Still encrypted at this point
     - **Service Layer Logging**: MEDIUM RISK - Decrypted data present, but limited exposure in controlled environment
     - **Log Access**: CONTROLLED - Only authorized personnel in controlled environment
   - **Mitigation Priority**: **MEDIUM** - Implement selective sanitization for service-layer logging as defense-in-depth, but request encryption provides primary protection at HTTP layer
4. **Database Configuration Security Vulnerabilities — REEVALUATED FOR CONTROLLED ENVIRONMENT**
   - **File Affected**: `apps/app/src/config/database.config.ts` (Lines: 8-16)
   - **Risk**: **MEDIUM** (Downgraded from Critical for controlled deployment)
   - **CWE**: CWE-798 (Hard-coded Credentials), CWE-319 (Cleartext Transmission)
   - **CVSS Score**: 8.6 (High in public deployment) → **5.3 (Medium in controlled environment)**
   - **Vulnerable Configuration**:
     ```
     export const baseDbConfig: DataSourceOptions = {
     ```

```
  host: process.env.DB_HOST || "localhost",          // Default
 ↪ fallback
  username: process.env.DB_USERNAME || "postgres",    // Default
 ↪ credentials
  password: process.env.DB_PASSWORD || "postgres",    // Weak default
 ↪ password
  ssl: process.env.DB_SSL === "true" ? {
    rejectUnauthorized: false                          // Insecure
 ↪ SSL config
  } : false,
  logging: process.env.NODE_ENV !== "production",     // May log
 ↪ sensitive queries
};
```

- **Issues**:
  - Default credentials: postgres/postgres in production
  - Weak SSL configuration: `rejectUnauthorized: false` allows MITM attacks
  - Query logging: SQL queries with sensitive data logged in development
  - Configuration exposure: Default values reveal production configuration
- **Impact (Controlled Environment Context)**:
  - **MITIGATED BY DEPLOYMENT**: Database runs in same isolated network (Kubernetes cluster)
  - **MITIGATED BY DEPLOYMENT**: Network-level access controls prevent external database access
  - **MITIGATED BY DEPLOYMENT**: Service mesh provides encrypted communication within cluster
  - **REMAINING RISK**: Default credentials should be changed per security best practices
  - **REMAINING RISK**: `rejectUnauthorized: false` reduces defense-in-depth
  - **REMAINING RISK**: Query logging may expose encrypted key data in non-production environments
- **Mitigation Priority**: **MEDIUM** - Update to enforce environment variables and remove defaults for production hardening, but network isolation provides primary protection

5. **Missing API Endpoint Access Controls — REEVALUATED FOR CONTROLLED ENVIRONMENT**
   - **Files**: All controller endpoints
   - **Risk Level**: **MEDIUM** (Downgraded from Critical for controlled deployment)
   - **CWE**: CWE-284 (Improper Access Control)
   - **CVSS Score**: 8.8 (High in public deployment) → **5.4 (Medium in controlled environment)**
   - **Description**: No authorization checks exist for sensitive operations:

```
// Anyone can generate keys for any identifier
@Post("generate")
generateKey(@Body() body: GenerateRequestDto | EncryptedPayloadDto) {
  return this.appService.generateKey(body);
}

// Anyone can sign credentials with any identifier
@Post("sign/vc/:type")
async signVC(@Param("type") type: SignType, @Body() body:
 ↪ SignRequestDto) {
```

```
    return await this.appService.signVC(type, body);
  }
```

- **Impact (Controlled Environment Context)**:
  - **MITIGATED BY DEPLOYMENT**: Kubernetes NetworkPolicies restrict access to authorized services only
  - **MITIGATED BY DEPLOYMENT**: Service mesh identity provides implicit service-level authentication
  - **MITIGATED BY ARCHITECTURE**: Request encryption requires shared secret, providing implicit authentication
  - **REMAINING RISK**: Any service with the shared encryption key can perform any operation
  - **REMAINING RISK**: No fine-grained authorization (e.g., which service can access which identifiers)
  - **REMAINING RISK**: Limited audit trail for compliance requirements
- **Mitigation Priority**: **MEDIUM** - Consider implementing service-level RBAC for defense-in-depth and audit trails, but network and encryption controls provide baseline protection

7.3 Medium-Risk Issues

6. **CORS Configuration Weaknesses**
   - **Files Affected**: `apps/app/src/main.ts` (Line: 18-22), `apps/app/src/config/cors.config.ts` (Lines: 20-27)
   - **Risk**: **MEDIUM**
   - **CWE**: CWE-346 (Origin Validation Error)
   - **CVSS Score**: 6.1 (Medium)
   - **Configuration Issues**:

```
// VULNERABLE: Dangerous fallback behavior
export const corsConfig: CorsConfig = {
  enabled: process.env.CORS_ENABLED === "true",
  options: process.env.CORS_ORIGINS ? {
    origin: process.env.CORS_ORIGINS.split(",").map(origin =>
↪   origin.trim()),
    // No origin validation
  } : undefined, // Falls back to allowing ALL origins
};
```

   - **Impact**: Cross-origin attacks, malicious website access, CSRF vulnerabilities
   - **Mitigation**: Implement origin validation and secure fallback behavior

7.4 Input Validation Analysis

EXCELLENT: Comprehensive input validation implemented with class-validator decorators including:

- String length validation
- Array size limits preventing buffer overflow
- Regular expression pattern matching
- Type safety through TypeScript and DTOs

7.5 Rate Limiting and DoS Protection Analysis

7. **Insufficient Rate Limiting Implementation**

- **Current Protection**: Limited to key decryption attempts only (3 attempts, 15-minute cooldown)
- **Missing Protection**:
  - No rate limiting on `/generate` endpoint (unlimited key generation)
  - No limits on `/sign/vc/*` endpoints (excessive signing operations)
  - No global per-IP or per-service limits
- **Risk**: Resource exhaustion, database connection pool exhaustion, memory exhaustion
- **Recommended Fix**: Implement comprehensive rate limiting with ThrottlerModule

7.6 Comparison with Previous Security Assessment

7.6.1 Status Updates from Previous Assessment (October 7, 2025)

MAINTAINED EXCELLENT STATUS:

- **Cryptographic Implementations**: MAINTAINED - Secure algorithms and best practices
- **Input Validation**: EXCELLENT DISCOVERY - Comprehensive class-validator implementation

PERSISTENT CRITICAL ISSUES (No Progress Since Previous Assessment):

- **Authentication/Authorization**: REMAINS CRITICAL - No authentication framework implemented
- **Error Handling**: REMAINS CRITICAL - Stack trace exposure unchanged
- **Database Security**: REMAINS CRITICAL - Default credentials still present
- **Cryptographic Key Material Logging Risk**: REMAINS CRITICAL - Logger exposes sensitive material

NEW CRITICAL SECURITY ISSUE IDENTIFIED:

- **NEW**: Missing API Endpoint Access Controls - Enhanced analysis of authorization gaps

DEPENDENCY SECURITY DEGRADATION:

- **Validator Package**: New moderate vulnerability identified (GHSA-9965-vmph-33xx)

7.6.2 Risk Level Progression

Security Domain	Previous Status	Reevaluated Status (Controlled Env
Overall Security Grade	D+ (Critical)	**B- (Acceptable)**
Authentication & Authorization	Critical	**Medium**
Error Handling & Information Disclosure	Critical	**Low**
Database Configuration	Critical	**Medium**
Logging Security	Critical	**Medium**
API Access Controls	Critical	**Medium**
Input Validation	Unknown	Excellent
CORS Configuration	Medium	Medium
Rate Limiting	Partial	Partial
Dependency Security	Excellent (0 vulnerabilities)	Moderate (1 vulnerability)
License Compliance	Excellent	Good (AGPL compliance required)

Context-Aware Assessment: When evaluated in the context of:

- **Controlled Environment**: Isolated Kubernetes cluster with NetworkPolicies

- **Non-Public Deployment**: Internal service-to-service communication only
- **Request Encryption**: Sensitive payloads encrypted before transmission
- **Service Mesh**: mTLS provides transport security between services

The security posture is **significantly better** than generic public-facing deployment. The previous critical issues are effectively mitigated by architectural and deployment controls, though defense-in-depth improvements are still recommended.

8 Security Recommendations (Updated for Controlled Environment Deployment)

8.1 High Priority Actions (HIGH - Within 1-2 Weeks)

Note: Previous "Critical" recommendations have been downgraded based on deployment context. The following recommendations focus on defense-in-depth improvements and best practices.

1. **Fix Dependency Vulnerability - STILL REQUIRED**

```
# Fix validator vulnerability
npm audit fix
npm update validator

# Verify vulnerability resolution
npm audit --audit-level=moderate
```

   - **Priority**: HIGH - While the service is in a controlled environment, keeping dependencies up-to-date is critical
   - **Impact**: Prevents potential exploitation if service is ever exposed or environment changes

8.2 Medium Priority Actions (MEDIUM - Within 2-4 Weeks - Defense-in-Depth)

2. **Implement Selective Cryptographic Logging Sanitization** (Downgraded from Critical):

```
const SENSITIVE_FIELDS = [
  'privatekey', 'publickey', 'secrets', 'password', 'token', 'signature',
  'privatekeymultibase', 'publickeymultibase', 'privatekeyjwk',
   ↪ 'publickeyjwk',
  'd', 'x', 'y', 'p', 'q', 'dp', 'dq', 'qi', 'key', 'secret',
   ↪ 'credential',
  'encryptedprivatekey', 'salt', 'iv', 'authtag', 'encrypted',
   ↪ 'decrypted',
  'jwk', 'jws', 'jwt', 'challenge', 'nonce'
];

const sanitizeMetadata = (obj: any): any => {
  if (!obj || typeof obj !== 'object') return obj;
  const sanitized = JSON.parse(JSON.stringify(obj));

  const sanitizeRecursive = (target: any): void => {
    Object.keys(target).forEach(key => {
```

```
      const lowerKey = key.toLowerCase();

      if (SENSITIVE_FIELDS.some(field => lowerKey.includes(field))) {
        target[key] = '[REDACTED]';
      } else if (typeof target[key] === 'object' && target[key] !== null)
↪  {
        sanitizeRecursive(target[key]);
      }
    });
  };

  sanitizeRecursive(sanitized);
  return sanitized;
};
```

- **Context**: Request encryption already protects HTTP layer logging
- **Benefit**: Additional protection for service-layer logging as defense-in-depth
- **Priority**: MEDIUM - Already protected at HTTP layer, this adds extra safety

3. **Environment-Based Error Sanitization** (Downgraded from Critical):

```
const sanitizedStack = process.env.NODE_ENV === 'production' ? undefined :
↪  exception.stack;
const sanitizedMessage = process.env.NODE_ENV === 'production'
  ? 'Internal server error'
  : message;

logError(`Error ${status}: ${sanitizedMessage}`, {
  type: process.env.NODE_ENV === 'production' ? 'Error' : type,
  path: request.url,
  method: request.method,
  stack: sanitizedStack,
});
```

- **Context**: In controlled environment, detailed errors aid debugging with minimal security risk
- **Benefit**: Balance between security and operational visibility
- **Priority**: MEDIUM - Consider implementing for production, but keep detailed errors for development/staging

4. **Secure Database Configuration** (Downgraded from Critical):

```
export const baseDbConfig: DataSourceOptions = {
  type: "postgres",
  // Remove ALL default fallbacks - fail fast if not configured
  host: process.env.DB_HOST || (() => {
    throw new Error('DB_HOST environment variable is required')
  })(),
  port: parseInt(process.env.DB_PORT || "5432"),
  username: process.env.DB_USERNAME || (() => {
    throw new Error('DB_USERNAME environment variable is required')
  })(),
  password: process.env.DB_PASSWORD || (() => {
    throw new Error('DB_PASSWORD environment variable is required')
```

```
  })(),
  database: process.env.DB_NAME || (() => {
    throw new Error('DB_NAME environment variable is required')
  })(),
  entities: [EncryptedKey],
  logging: false, // Disable query logging to prevent sensitive data
↪  exposure
  ssl: process.env.DB_SSL === "true" ? {
    rejectUnauthorized: true,    // Enforce certificate validation
    ca: process.env.DB_SSL_CA,
    cert: process.env.DB_SSL_CERT,
    key: process.env.DB_SSL_KEY,
  } : false,
};
```

- **Context**: Network isolation already prevents external database access
- **Benefit**: Follows security best practices and prevents accidental misconfigurations
- **Priority**: MEDIUM - Good practice but network controls provide primary protection

5. **Consider Service-Level Authentication for Defense-in-Depth** (Downgraded from Critical):

```
@Injectable()
export class ApiKeyAuthGuard implements CanActivate {
  private readonly validApiKeys = process.env.API_KEYS?.split(',') || [];

  canActivate(context: ExecutionContext): boolean {
    if (this.validApiKeys.length === 0) {
      throw new Error('API_KEYS environment variable must be configured');
    }

    const request = context.switchToHttp().getRequest();
    const apiKey = request.headers['x-api-key'];

    if (!apiKey) {
      throw new UnauthorizedException('API key required');
    }

    if (!this.validApiKeys.includes(apiKey)) {
      throw new UnauthorizedException('Invalid API key');
    }

    return true;
  }
}

@UseGuards(ApiKeyAuthGuard)
@Controller()
export class AppController { /* ... */ }
```

- **Context**: Network policies and request encryption already provide authentication
- **Benefit**: Adds defense-in-depth and enables better audit trails

- **Priority**: MEDIUM - Consider for enhanced compliance and audit requirements

8.3 Lower Priority Actions (LOW - Within 1-2 Months - Best Practices)

6. **Comprehensive Rate Limiting**: Implement global and endpoint-specific limits using ThrottlerModule
   - **Context**: Failed decryption rate limiting already exists
   - **Benefit**: Prevents resource exhaustion from compromised internal service
   - **Priority**: LOW - Risk is limited in controlled environment
7. **Enhanced CORS Security**: Implement origin validation and secure fallbacks
   - **Context**: Internal service-to-service communication, not browser-based
   - **Benefit**: Minimal benefit for non-browser clients
   - **Priority**: LOW - Not applicable for service-to-service architecture
8. **Enhanced Security Headers**: Implement comprehensive security headers using Helmet
   - **Context**: Headers primarily benefit browser-based clients
   - **Benefit**: Minimal for service-to-service communication
   - **Priority**: LOW - Consider if service is ever accessed by browsers
9. **Security Monitoring and Alerting**: Add security event logging and monitoring
   - **Benefit**: Enhanced visibility and audit capabilities
   - **Priority**: LOW - Consider for compliance requirements
10. **Automated Security Testing**: Implement security testing in CI/CD pipeline
    - **Benefit**: Continuous security validation
    - **Priority**: LOW - Good practice for ongoing security posture
11. **AGPL-3.0 Compliance Documentation**: Create formal compliance procedures
    - **Benefit**: Legal compliance
    - **Priority**: LOW - Depends on licensing strategy

8.4 Security Testing

8.4.1 Recommended Testing

1. **Static Analysis**: ESLint security rules, Semgrep
2. **Dependency Scanning**: Automated `npm audit` integration
3. **Penetration Testing**: Regular security assessments focused on:
   - Authentication bypass attempts
   - Input validation fuzzing
   - SQL injection testing
   - Rate limiting validation
   - Error message analysis
   - CORS policy testing

8.5 Security Checklist (Updated October 31, 2025 - Controlled Environment Deployment)

8.5.1 COMPLETED ITEMS

☒ SBOM analysis with comprehensive license distribution assessment
☒ Cryptographic implementations follow best practices
☒ Input validation comprehensive implementation with class-validator
☒ Rate limiting implemented for failed decryption attempts
☒ SQL injection protection via TypeORM
☒ Request encryption implemented (AES-256-GCM)
☒ Network isolation via Kubernetes deployment

☒ Service mesh with mTLS enabled

8.5.2 HIGH PRIORITY PENDING ITEMS (Within 1-2 weeks)

☐ **HIGH**: Fix dependency vulnerability (validator package) - **REQUIRED**

8.5.3 MEDIUM PRIORITY ITEMS (Within 2-4 weeks - Defense-in-Depth Recommended)

☐ **MEDIUM**: Implement selective cryptographic logging sanitization
☐ **MEDIUM**: Add environment-based error sanitization
☐ **MEDIUM**: Secure database configuration (remove defaults, enforce SSL)
☐ **MEDIUM**: Consider service-level authentication for audit trails
☐ **MEDIUM**: Implement API endpoint access controls for defense-in-depth

8.5.4 LOW PRIORITY ITEMS (Within 1-2 months - Best Practices)

☐ **LOW**: Implement global rate limiting and endpoint-specific throttling
☐ **LOW**: Enhanced CORS security (if needed for browser access)
☐ **LOW**: Security headers implementation (if needed for browser access)
☐ **LOW**: AGPL-3.0 compliance documentation
☐ **LOW**: Security monitoring and alerting system
☐ **LOW**: Automated security testing in CI/CD
☐ **LOW**: Comprehensive security documentation
☐ **LOW**: Incident response procedures

9 Reporting Vulnerabilities

9.1 Security Contact

For security-related issues, please contact:

- **Email**: christian.fries@eecc.de
- **Response Time**: 48 hours for acknowledgment
- **Disclosure**: Coordinated disclosure preferred

9.2 Reporting Guidelines

1. **Do Not** create public GitHub issues for security vulnerabilities
2. **Include** detailed reproduction steps and impact assessment
3. **Provide** suggested fixes if available
4. **Allow** reasonable time for fixes before public disclosure

9.3 Supported Versions

Version	Supported	Security Status (Controlled Environment)	Security Status (Public Deployment)
1.6.1	Yes	Acceptable - Defense-in-depth improvements recommended	Critical Issues - Not recommended for p
< 1.6	No	Not Supported	Not Supported

Last Updated: October 31, 2025 **Reevaluation Date**: October 31, 2025 (Updated for controlled environment deployment) **Security Review**: **MEDIUM** - Acceptable for controlled

environment deployment with defense-in-depth improvements recommended **Next Review Date**: After dependency fix (within 2 weeks), then quarterly review recommended **Dependency Scan**: **MODERATE** (1 vulnerability identified - requires fix) - Last checked: October 31, 2025 **SBOM Analysis**: **GOOD** (AGPL compliance required, 99.6% permissive dependencies) - Last checked: October 31, 2025 **Code Security Scan**: **MEDIUM** (30 issues identified, 0 critical in controlled environment context) - Last checked: October 31, 2025

Deployment Context:

- Controlled Kubernetes environment with NetworkPolicies
- Internal service-to-service communication only (NOT publicly accessible)
- Request encryption enabled (AES-256-GCM)
- Service mesh with mTLS
- Restricted log access

Classification: CONFIDENTIAL - Internal Security Assessment **Overall Security Rating**: **B- (Acceptable for Controlled Environment - Defense-in-Depth Improvements Recommended) Production Ready**: **YES** - For controlled environment deployment with following conditions:

- Network isolation enforced
- Request encryption enabled
- Dependency vulnerability should be fixed within 1-2 weeks
- Defense-in-depth improvements recommended but not blocking

Report compiled by: AI Security Crew working for Christian Fries <christian.fries@eecc.de> **Contact for Issues**: christian.fries@eecc.de **Copyright**: European EPC Competence Center GmbH, 2025